Refinement levels

1 Workload specification
kernels · training step · prefill/decode
shapes, precision, error bounds, SLOs

2 Computation graph
operators, producer–consumer edges

3 Operator transformation
fusion, split reductions, streaming

4 Tensor execution mapping
tiles, layouts ↔ registers, shared memory

5 Instruction scheduling
loads, MMA, barriers ↔ SW/HW boundary

6 System architecture
SMs, tensor units, memory, HBM, NoC

7 Chip microarchitecture
banks, ports, queues, arbitration

software-stack structure chip + software

Each level adds one step of detail to both the software stack and the chip; correctness composes back to level 1.

next level

LLM agent

- selects the next refinement step
- draws, adapts or generates templates
- diagnoses evidence: design, model, formulation or search policy

draw

return

Template population

formulation patterns, not designs:
template · scope Γ · evidence · signature



structurally diverse; admitted only once certified

authors

Template = contract for one refinement step

decisions X (software + chip) · search constraints C
preconditions Pre · backward map π · certificate (R, ρ) · scope Γ

evidence:
costs, discrepancies

C

Pre

Optimization solver

joint software + chip search
within constraints C
→ assignments, bounds

candidates

Verification gate

template soundness: once
instance check: every
candidate, against Pre

admitted

Evaluator

paired program + chip
PPA: analytic model →
simulation → hardware

counterexample → repair template

Certified design

one configuration across all 7 layers + proof

Repeat: refine → generate template → optimize → gate → evaluate → learn